

SLAM-ASR — Video Presentation Cheat-Sheet

Target 6:00 (range 5–7). One long take. QuickTime ⇒ File ⇒ New Screen Recording. Both notebooks open with outputs baked in. Cursor size boosted. Font zoom 120 %.

Time	Segment	On-screen
0:00–0:20	Hook	Repo landing page
0:20–1:00	Problem + context	Notebook §4 (manifest stats)
1:00–1:45	Architecture	README §1 or Section 9 output
1:45–4:00	Working demo ★	Inference notebook
4:00–5:00	Results	Notebook §11 table
5:00–5:30	Mode-collapse	Sample predictions
5:30–6:00	Wrap-up	Repo landing page

0:00 – 0:20 · Hook (20 s)

SHOW → [GitHub repo landing page](#).

SAY → “Hi, I’m Ravi Teja Kothuru. For our IIIT-Delhi Building the Future of Voice & Audio course, I built **SLAM-ASR** — an end-to-end speech recognition system for **Hindi-English code-switched** technical lectures. The next six minutes: what I built, a live demo, and — most importantly — the honest story of what worked and what didn’t.”

0:20 – 1:00 · Problem + context (40 s)

SHOW → [Section 4 of training notebook](#) — `manifest_statistics()` output.

SAY → “The MUCS 2021 Subtask-2 dataset is Indian technical-education recordings — Linux tutorials, Python lectures. A typical utterance sounds like: ‘ab hum terminal kholenge aur is command run karenge’ — half Hindi, half English, script-switched mid-sentence. Every second line switches script, one in four blindtest words is unseen during training. That combination breaks every off-the-shelf ASR. My project builds one that doesn’t.”

1:00 – 1:45 · Architecture (45 s)

SHOW → [README §1 ASCII diagram](#) OR [Section 9](#) “trainable / total = 23.97 M / 933 M” line.

SAY → “Three parts. First, a **frozen Whisper-base encoder** — 74 M pretrained params turning audio into features. Never touched. Second, a **small trainable projector** — 1-D conv + MLP into the LLM’s embedding space. Third, a **Qwen-2.5 1.5B decoder** loaded in 4-bit precision with LoRA adapters. Only 24 M params train — 2.6 % of the model — and the whole thing fits into a single free Kaggle T4 session.”

1:45 – 4:00 · Working demo ★ (2 min 15 s)

SHOW → [Inference notebook](#) — `slam_asr_inferencing_demo_transcribe.ipynb`.

SAY → “This inference notebook downloads the checkpoint, reconstructs the model on one GPU, and gives a one-line `transcribe()` function. Everything renders inline.

Scroll to Section 7 — 3 random test utterances. For each: **click play** on the audio (2 s), point at reference, point at hypothesis. After the 3rd, say: “All three predictions look almost identical. I’ll come back to that.”

Scroll to Section 8 — change `IDX = 42` to `IDX = 100`, re-run. Say: “New audio, new reference — same output. This is the crux of the findings.”

4:00 – 5:00 · Results table (60 s)

SHOW → [Section 11 of training notebook](#) — `pandas` comparison table.

SAY → “Four models on 300 test + 300 blindtest utterances. **Bi-LSTM+CTC**: WER 0.998 (near-empty output). **Whisper zero-shot**: 1.55 (hallucinates in silences). **Whisper fine-tuned**: 3.93 (prompt-format drift). **SLAM-ASR**: **2.06 test / 1.63 blindtest**. None would win a leaderboard — but they’re informatively different. Every model fails in a distinct way; that’s the real finding.”

5:00 – 5:30 · Mode-collapse analysis (30 s)

SHOW → [Sample predictions from slam_asr_outputs/predictions/test.jsonl](#).

SAY → “SLAM-ASR is under-trained. Only 236 optimizer steps on 3 h vs the paper’s 400 GPU-hours. The projector hasn’t learned input-dependent embeddings yet — it emits the same seed sentence for every input. This is **mode collapse**, a well-documented under-training pathology. The architecture is sound; it needs ~200× more compute. Every SLAM-ASR prediction is still **coherent, bilingual, script-mixed** — more than the other three models produce. Most promising of the four for scale-up.”

5:30 – 6:00 · Wrap-up + GitHub URL (30 s)

SHOW → [GitHub repo landing page](#). Hold on screen.

SAY → “Everything is at github.com/kraviteja95/slam-asr-code-switched-speech-recognition. Full report in `docs/`. Both Kaggle notebooks runnable end-to-end. Trained checkpoint is committed, split into small parts — `REASSEMBLE.md` in each folder explains how. Thanks for watching.”

Before recording: Do-Not-Disturb ON · Notifications silenced · Notebooks open with outputs · Mic checked · 10-s test recording done. **While recording:** One long take · Don’t rush · 2-s pauses OK · Small verbal stumbles fine. **After:** Trim dead space · Export MP4 H.264 · ≤ 500 MB · Save as `docs/demo.mp4` · Verify in QuickTime + VLC before committing.